

Spécification et Validations de Circuits Électroniques

Rapport de Projet d'Ingénierie Dirigée par les Modèles (IDM)

Groupe L4

Ounchari imane

Ellaik fadwa

Bouffi douaa

Année 2025-2026

Table des matières

1	Introduction	4
2	Architecture et Méta-modèles	4
2.1	Le Catalogue de Composants	4
2.2	La Netlist (Circuit Logique)	4
2.3	Le Layout (Circuit Physique)	5
3	Syntaxes Concrètes et Éditeurs	6
3.1	Édition Textuelle du Catalogue (Xtext)	6
3.2	Édition Graphique de la Netlist et du Layout (Sirius)	7
3.2.1	Netlist	7
3.2.2	Layout	8
4	Validation et Sémantique	9
4.1	Validation du Catalogue	9
4.2	Validation du Layout (Non-chevauchement)	10
5	Génération et Export	10
5.1	Génération de BOM (Bill of Materials)	10
5.2	Génération SVG	11
6	Description du Rendu	12
7	Conclusion	12

Table des figures

1	Diagramme de classe du méta-modèle Catalogue	4
2	Diagramme de classe du méta-modèle Netlist	5
3	Diagramme de classe du méta-modèle Layout	6
4	Éditeur Xtext pour le Catalogue avec coloration syntaxique	7
5	Éditeur graphique Sirius pour la Netlist	8
6	Éditeur graphique Sirius pour le Layout (Vue PCB)	9
7	Validation en temps réel d'une contrainte de redondance invalide	10
8	Extrait du fichier BOM (CSV) généré	10
9	Tranformation java d'un circuit simple en format svg	11
10	Tranformation java d'un circuit avancé en format svg	11

1 Introduction

L'électronique embarquée moderne impose des contraintes de fiabilité et de miniaturisation croissantes. La conception de tels systèmes nécessite des outils rigoureux pour garantir, dès la phase de design, le respect de règles physiques et logiques.

Ce projet s'inscrit dans cette démarche en proposant une suite logicielle basée sur l'Ingénierie Dirigée par les Modèles (IDM) pour la spécification et la validation de circuits imprimés (PCB). L'objectif est de fournir à l'ingénieur électronicien un environnement complet permettant de définir un catalogue de composants, de concevoir un circuit logique (Netlist) et de réaliser son implantation physique (Layout), tout en assurant la cohérence et la validité de l'ensemble via des mécanismes automatisés.

Ce rapport détaille les choix de conception, l'architecture des méta-modèles, les éditeurs (graphiques et textuels) développés, ainsi que les mécanismes de validation et de génération implémentés.

2 Architecture et Méta-modèles

Pour répondre au principe de séparation des préoccupations, nous avons choisi de diviser le système en trois méta-modèles distincts mais interconnectés.

2.1 Le Catalogue de Composants

Le méta-modèle Catalogue définit les briques de base utilisables dans les circuits. Il sépare la définition logique (nom, fabricant) de la définition physique (empreinte).

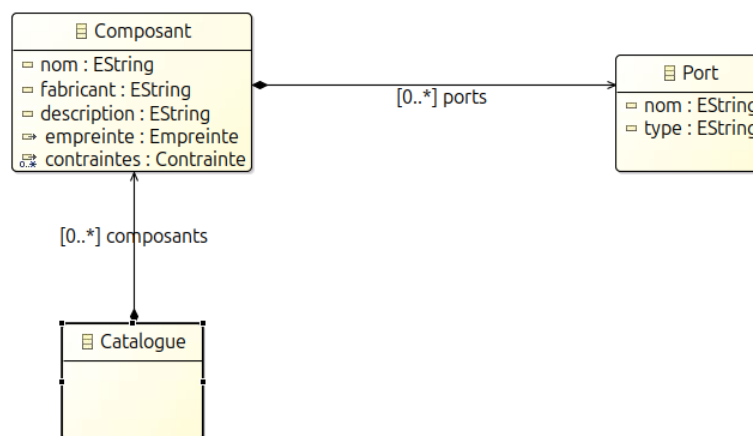


FIGURE 1 – Diagramme de classe du méta-modèle Catalogue

Comme illustré par la Figure 1, un **Composant** possède des **Ports** et une **Empreinte**. Nous avons intégré un système de contraintes flexible (composite pattern) permettant de définir des règles logiques (ET, OU, NON) et géométriques directement dans le modèle.

2.2 La Netlist (Circuit Logique)

Le méta-modèle **Netlist** représente le schéma électrique. Il fait référence aux composants du catalogue sans les dupliquer.

- **Circuit** : La racine, contenant les instances et les connexions.

- **InstanceComposant** : Une occurrence d'un composant du catalogue (ex : "R1" est une instance de "Resistance 10k").
- **Connexion** : Relie deux ou plusieurs ports d'instances différentes.

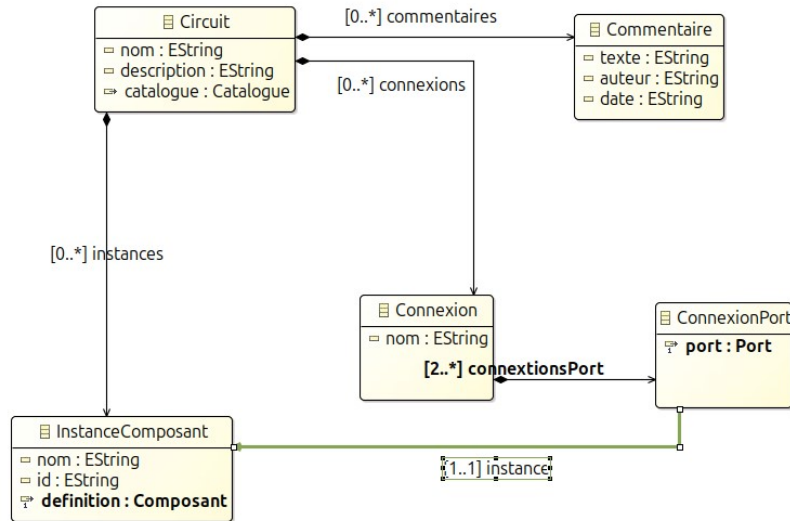


FIGURE 2 – Diagramme de classe du méta-modèle Netlist

Ce modèle assure l'indépendance entre la logique du circuit et son placement physique.

2.3 Le Layout (Circuit Physique)

Le méta-modèle Layout décrit la réalité physique de la carte.

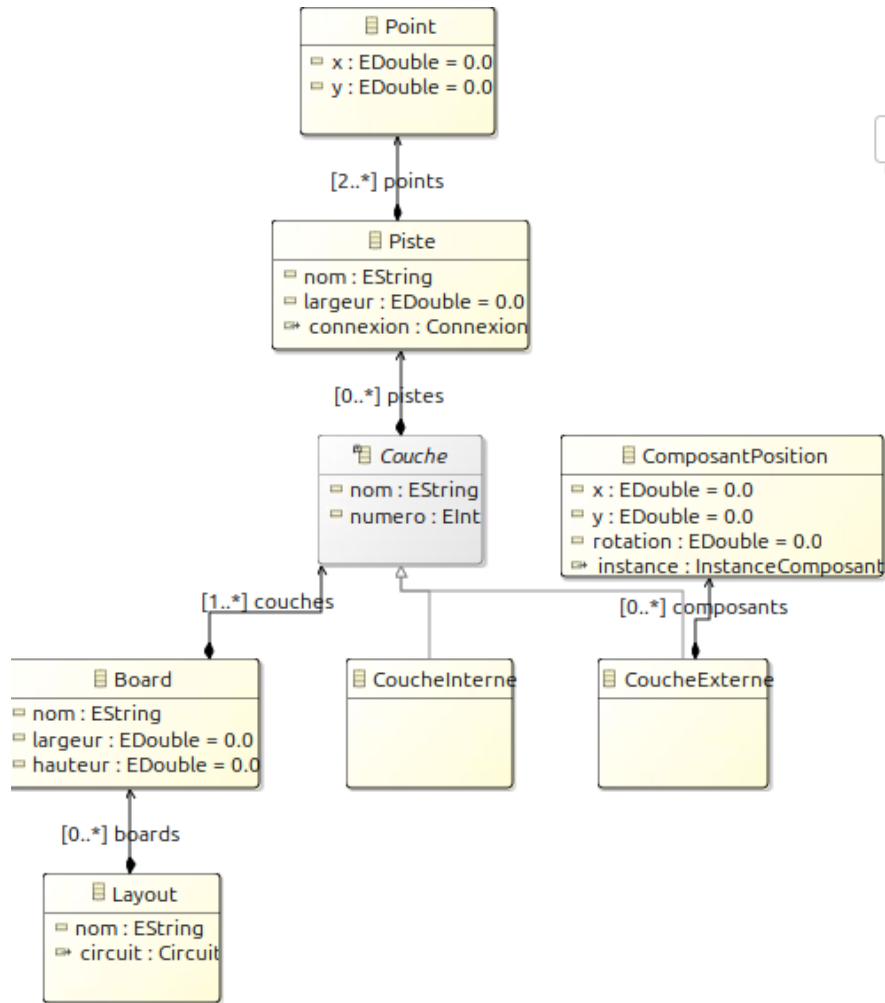


FIGURE 3 – Diagramme de classe du méta-modèle Layout

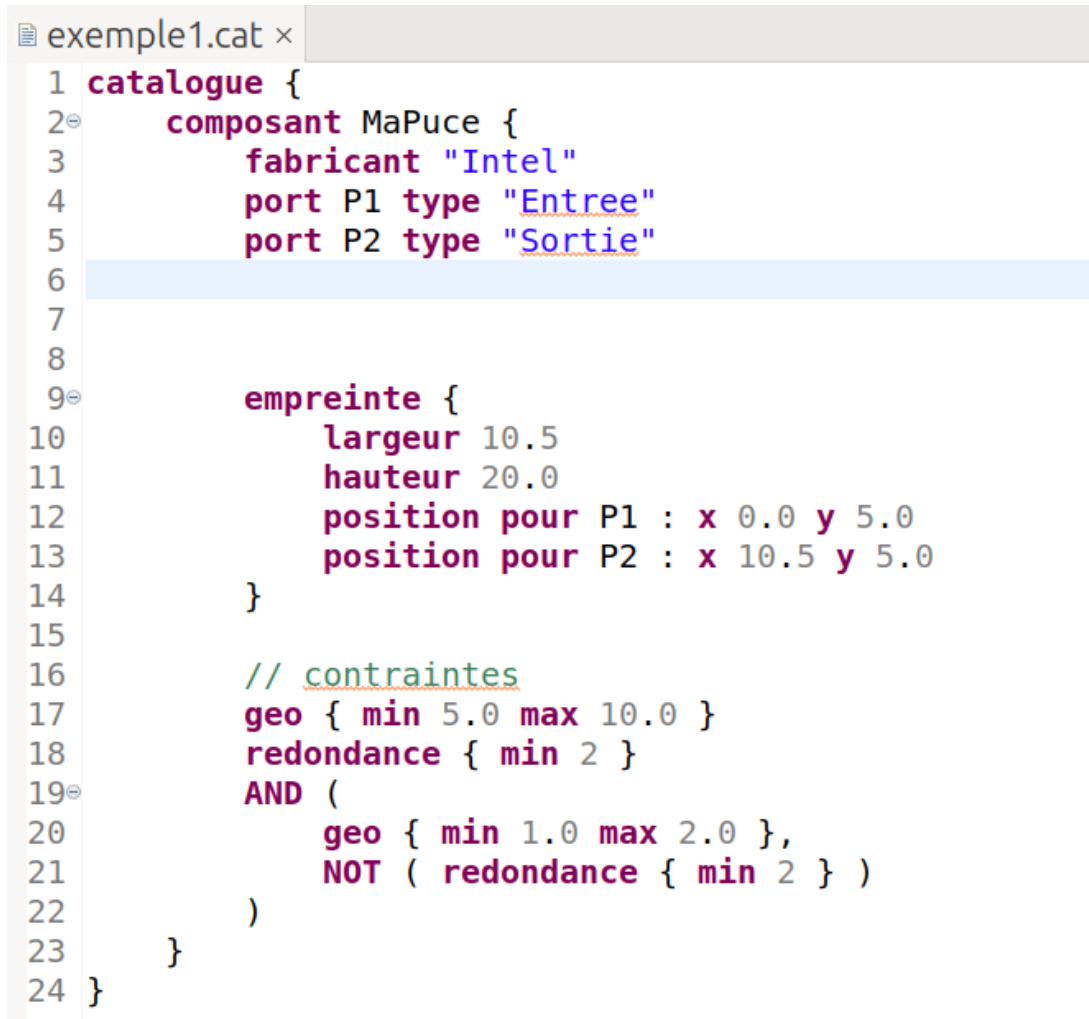
Il introduit les concepts de Board (la carte), de Couche (interne ou externe) et de Piste (Figure 3). La position des composants (ComposantPosition) est définie par des coordonnées précises (x, y) et une rotation, tout en gardant un lien fort vers l'InstanceComposant de la Netlist pour garantir la traçabilité.

3 Syntaxes Concrètes et Éditeurs

Conformément aux exigences, nous avons développé des éditeurs adaptés à la nature de chaque modèle.

3.1 Édition Textuelle du Catalogue (Xtext)

Pour le catalogue, une syntaxe textuelle a été privilégiée car elle permet une saisie rapide et efficace des métadonnées et des contraintes.



```
1 catalogue {
2     composant MaPuce {
3         fabricant "Intel"
4         port P1 type "Entree"
5         port P2 type "Sortie"
6
7
8
9     empreinte {
10         largeur 10.5
11         hauteur 20.0
12         position pour P1 : x 0.0 y 5.0
13         position pour P2 : x 10.5 y 5.0
14     }
15
16     // contraintes
17     geo { min 5.0 max 10.0 }
18     redondance { min 2 }
19     AND (
20         geo { min 1.0 max 2.0 },
21         NOT ( redondance { min 2 } )
22     )
23 }
24 }
```

FIGURE 4 – Éditeur Xtext pour le Catalogue avec coloration syntaxique

L'éditeur développé avec Xtext (Figure 4) offre l'autocomplétion, la coloration syntaxique et la validation à la volée. Nous avons dû implémenter un `QualifiedNameProvider` personnalisé pour gérer l'identification des éléments via leur attribut nom (français) plutôt que `name` par défaut.

3.2 Édition Graphique de la Netlist et du Layout (Sirius)

Pour la Netlist et le Layout, une représentation graphique est indispensable pour visualiser la topologie.

3.2.1 Netlist

Le point de vue Sirius pour la Netlist permet de relier visuellement les composants. L'outil de création de connexion a été configuré pour empêcher les connexions invalides (port à lui-même).

- ▼ <platform:/resource/fr.n7.idm.netlist.design/description/netlist.odesign>
 - ▼ NetlistViewpoint
 - ▼ NetlistViewpoint
 - ▼ NetlistDiagram
 - ▼ Default
 - ▶ InstanceNode
 - ▶ ConnexionNode
 - ▶ ConnexionEdge
 - ▼ Section CreationTools
 - ▶ Node Creation Instance Composant
 - ▶ Node Creation Nœud de Connexion
 - ▶ Edge Creation Lier au Port
 - ▶ Delete Element Delete
- http://www.example.org/netlist
- http://www.example.org/catalogue

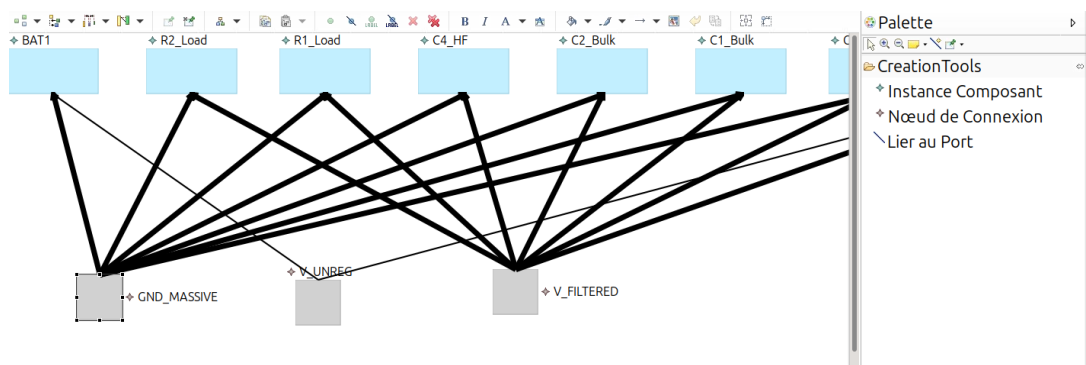
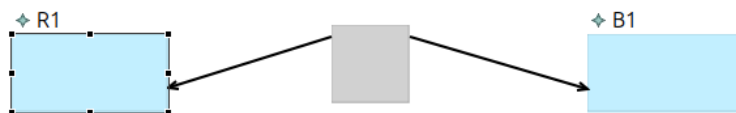


FIGURE 5 – Éditeur graphique Sirius pour la Netlist

3.2.2 Layout

L'éditeur Layout représente physiquement la carte (Figure 6). Les composants apparaissent sous forme de boîtes dont la taille dépend de leur empreinte (récupérée dynamiquement depuis le catalogue). Les différentes couches sont gérées via des conteneurs ou des filtres graphiques.

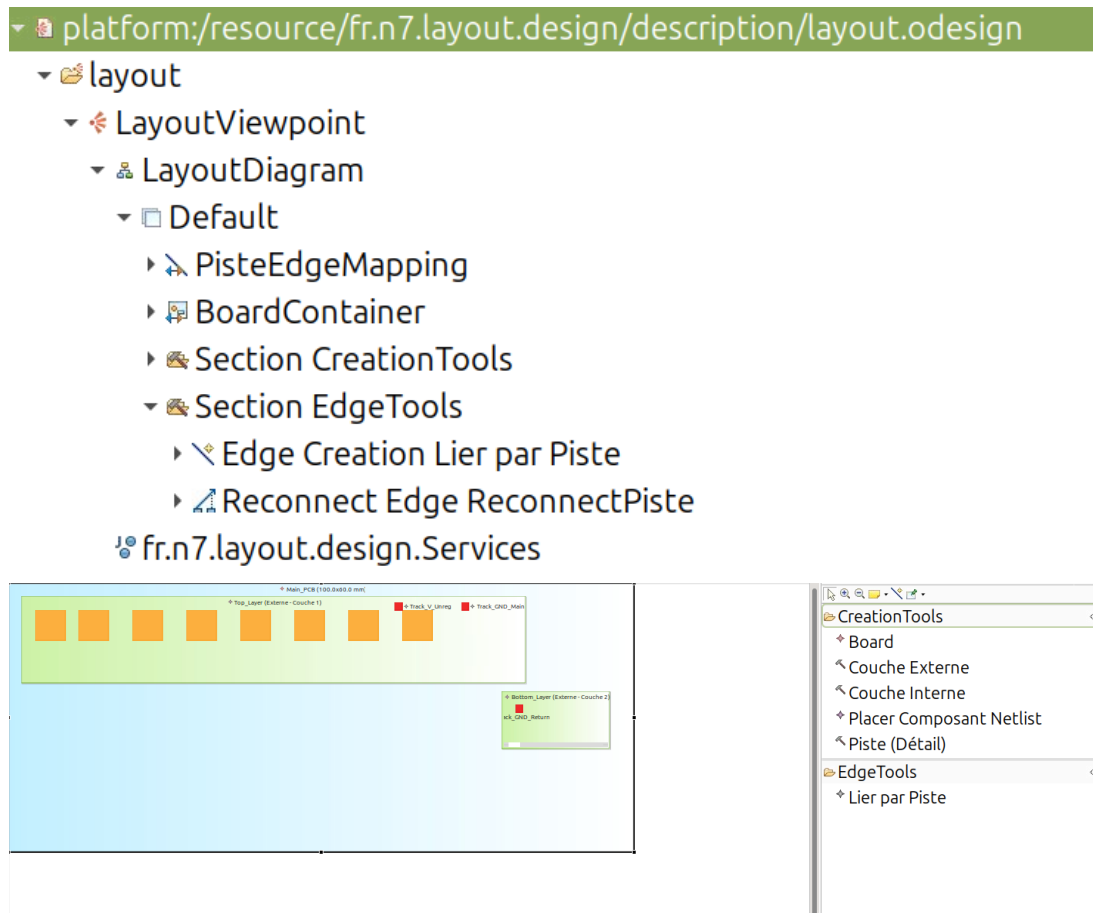


FIGURE 6 – Éditeur graphique Sirius pour le Layout (Vue PCB)

4 Validation et Sémantique

La robustesse du design repose sur des règles de validation strictes implémentées en Java (sans OCL, pour des raisons de performance et de complexité algorithmique).

4.1 Validation du Catalogue

Des règles métier vérifient la cohérence des définitions. Par exemple, une contrainte de redondance doit imposer au moins deux exemplaires d'un composant.

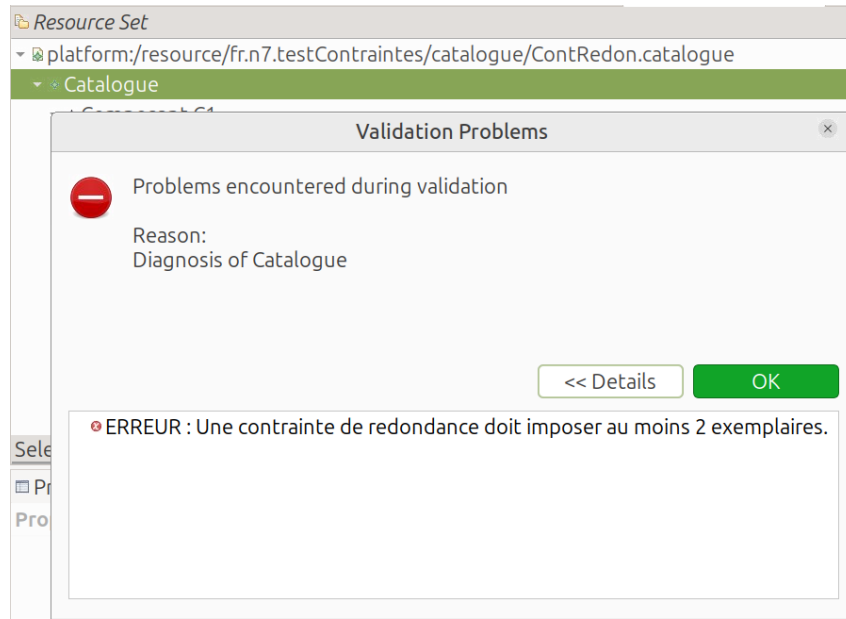


FIGURE 7 – Validation en temps réel d’une contrainte de redondance invalide

4.2 Validation du Layout (Non-chevauchement)

La contrainte la plus critique implémentée est la détection de chevauchement (Overlap) sur le PCB. L’algorithme utilise une approche AABB (Axis-Aligned Bounding Box). Le validateur :

1. Récupère la position (X, Y) de chaque composant dans le Layout.
2. Remonte via la Netlist jusqu’au Catalogue pour obtenir la largeur et la hauteur de l’empreinte.
3. Calcule les intersections géométriques.

Si deux composants se chevauchent, une erreur bloquante est levée dans l’éditeur.

5 Génération et Export

5.1 Génération de BOM (Bill of Materials)

Une transformation Model-to-Text (M2T) permet d’extraire la liste des composants nécessaires à la fabrication.

	Standard	Standard	Standard	Standard	Standard
1					
2	ID_Instance	Nom_Instance	Nom_Composant	Fabricant	Description
3	B1	B1	Batterie	Saft	Batterie Li-ion haute capacité
4	R1	R1	Resistance	Vishay	Résistance de précision
5					

FIGURE 8 – Extrait du fichier BOM (CSV) généré

Le fichier CSV généré (Figure 10) agrège les informations provenant de la Netlist (IDs d’instances) et du Catalogue (Fabricant, Description).

5.2 Génération SVG

Bien que non présentée visuellement ici, une chaîne de génération a été conçue pour exporter les couches du Layout au format vectoriel SVG. Cette transformation parcourt le modèle Layout et génère des balises `<rect>` pour les composants et `<path>` pour les pistes, permettant une visualisation dans n'importe quel navigateur web.



FIGURE 9 – Transformation java d'un circuit simple en format svg

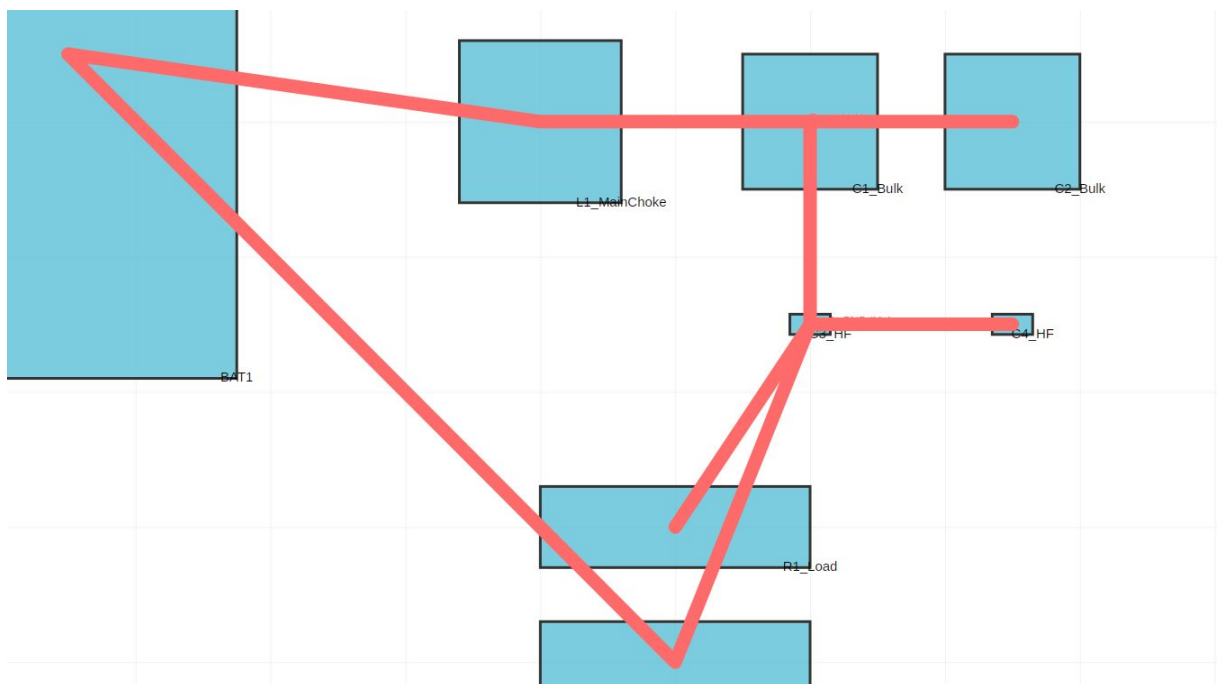


FIGURE 10 – Transformation java d'un circuit avancé en format svg

6 Description du Rendu

Le rendu Git est organisé en plusieurs projets Eclipse (*workspaces*) respectant les conventions de nommage :

- `fr.n7.idm.catalogue` : Méta-modèle et validateur Java du catalogue.
- `fr.n7.idm.catalogue.xtext` : Grammaire et éditeur textuel.
- `fr.n7.idm.netlist` : Méta-modèle de la netlist.
- `fr.n7.idm.netlist.design` : Spécification Sirius pour la netlist.
- `fr.n7.idm.layout` : Méta-modèle du layout et algorithmes géométriques.
- `fr.n7.idm.layout.design` : Spécification Sirius pour le PCB.

Tous les fichiers sources (`.ecore`, `.genmodel`, `.java`, `..xtext`, `.odesign`) sont inclus, ainsi que des exemples de modèles (`test.catalogue`, `monCircuit.layout`) permettant de reproduire les démonstrations.

7 Conclusion

Ce projet nous a permis de mettre en œuvre une chaîne de production logicielle complète basée sur l’IDM. La séparation en trois méta-modèles s’est avérée structurante, facilitant le travail parallèle.

Points forts :

- L’intégration réussie entre Xtext (texte) et Sirius (graphique) au sein du même environnement.
- La gestion complexe des références inter-modèles (Layout → Netlist → Catalogue).
- L’implémentation d’algorithmes de géométrie en Java pur connectés aux modèles EMF.

Difficultés rencontrées : La principale difficulté a résidé dans la configuration initiale des dépendances entre les plugins Eclipse et la gestion des proxies EMF lors du chargement de ressources multiples. L’adaptation de Xtext pour utiliser des noms en français (nom vs name) a également demandé une configuration spécifique du `QualifiedNameProvider`.

En conclusion, la suite réalisée répond au cahier des charges en offrant un environnement robuste pour la spécification et la validation précoce de circuits électroniques.